

Creación de
componentes

Proyecto final UD3



Carmen Campos Fernández

CONTENIDO

Fase 1: Creación y Empaquetado de la Librería de Controles (El Emisor).....	2
1. Crear el Proyecto Contenedor (La DLL).....	2
2. Componente A: El Visor de Progreso (UserControl)	2
3. Componente B: El Botón de Acción (CustomControl)	3
4. Coordinación de la Acción (La Tarea Principal).....	3
5. Empaquetado y Mapeo Profesional (La DLL)	3
Fase 2: Adaptación y Uso en tu Proyecto CRUD Existente.....	4
6. Guía de Compatibilidad y Referencia.....	4
7. Implementación en la Ventana Principal	4
ENTREGA.....	4
Instrucciones Detalladas de Entrega.....	4
Criterios de Evaluación	6



Creación y Distribución de Componentes DLL

Entrega individual

Este proyecto es la culminación de todo lo que has aprendido sobre la creación, empaquetado y uso de componentes reutilizables en WPF. Vas a mejorar tu proyecto CRUD anterior añadiéndole una funcionalidad visual profesional.

FASE 1: CREACIÓN Y EMPAQUETADO DE LA LIBRERÍA DE CONTROLES (EL EMISOR)

Debes crear un proyecto de librería de clases para contener **dos** componentes visuales, un UserControl y un CustomControl, que interactuarán entre sí de forma coordinada.

1. CREAR EL PROYECTO CONTENEDOR (LA DLL)

1. Crea un nuevo proyecto llamado **LibreriaVisualFinal** utilizando la plantilla **WPF Class Library** (Biblioteca de Clases de WPF).
2. **Guía Detallada de Compatibilidad (¡Paso Clave!)**
 - Tu proyecto CRUD utiliza la versión clásica de .NET (ej., .NET Framework 4.7.2). Tu librería debe usar el mismo marco.
 - Haz clic derecho en el proyecto **LibreriaVisualFinal** > "**Propiedades**".
 - En la pestaña "**Aplicación**", verifica el campo "**Marco de destino**" (Target Framework).
 - **Asegúrate de cambiarlo** a la versión de **.NET Framework 4.x** que esté instalada y sea compatible con tu proyecto CRUD (ej., 4.7.2 o 4.8).

2. COMPONENTE A: EL VISOR DE PROGRESO (USERCONTROL)

Este control, basado en un UserControl, será el receptor de la acción y mostrará el estado de la misma.

- **Nombre:** VisorProgreso (UserControl).
- **Diseño (XAML):** Debe contener:
 - Un **TextBlock** (para mostrar el mensaje de la última acción).
 - Un **ProgressBar** (Barra de Progreso) debajo (para mostrar un valor numérico de progreso).
- **Dependency Property (DP):**
 - Crea la DP **MensajeAccion** de tipo string. Enlaza internamente el TextBlock a esta DP, con un valor por defecto de: "Esperando interacción..."

- Crea la DP **ProgresoValor** de tipo double. Enlaza internamente la propiedad Value del ProgressBar a esta DP (asumiendo que Minimum=0 y Maximum=100).

3. COMPONENTE B: EL BOTÓN DE ACCIÓN (CUSTOMCONTROL)

Este control es altamente visual y funcionará como el emisor de la acción, heredando de la funcionalidad nativa de un Button.

- **Nombre:** BotonVisual (CustomControl), heredando de **Button**.
- **Clase (.cs):** Implementa el **Constructor Estático** (static BotonVisual()) para que busque su estilo en el **Generic.xaml**.
- **Lógica de Interacción (.cs):** Sobreescribe **OnApplyTemplate()**.
 - Suscribe el evento **Click** del botón.
 - **Detalles de la Lógica del Color:** En el manejador del Click, debes realizar la siguiente tarea:
 - **Opción Sencilla (Alternar):** Define 3 colores fijos (ej., Azul, Verde, Amarillo) y utiliza una variable interna (como un contador o un if/else) para cambiar el Background del BotonVisual entre esos 3 colores en cada clic.
 - **Opción Avanzada (Aleatorio):** Crea una instancia de Random y genera tres valores de byte (entre 0 y 255) para las componentes Rojo, Verde y Azul (R, G, B). Luego usa System.Windows.Media.Color.FromRgb(r, g, b) para crear un color dinámico y aplicarlo al Background de tu control.

4. COORDINACIÓN DE LA ACCIÓN (LA TAREA PRINCIPAL)

Debes lograr que ambos controles trabajen juntos cuando el usuario haga clic:

1. Al hacer clic en el **BotonVisual** (donde cambia el color), debes actualizar:
 - La DP **MensajeAccion** del VisorProgreso para mostrar: *"Acción lanzada a las [Hora:Minutos:Segundos]"* (Debes obtener la hora actual usando DateTime.Now).
 - La DP **ProgresoValor** del VisorProgreso para mostrar un valor **aleatorio** entre 0 y 100.

5. EMPAQUETADO Y MAPEO PROFESIONAL (LA DLL)

1. **Mapeo XMLNS:** En la librería LibreriaVisualFinal, define el mapeo de *namespace* (Paso 4 de los apuntes):

C#

```
[assembly: XmlnsDefinition("http://miscontroles.com/final", "LibreriaVisualFinal")]
```

2. **Generación de la DLL:** Compila la solución para generar el archivo **LibreriaVisualFinal.dll**.

FASE 2: ADAPTACIÓN Y USO EN TU PROYECTO CRUD EXISTENTE

6. GUÍA DE COMPATIBILIDAD Y REFERENCIA

1. Abre tu proyecto CRUD existente (UD2).
2. Haz clic derecho en "**Referencias**" (References) del proyecto.
3. Selecciona "**Agregar Referencia...**" (Add Reference).
4. Ve a la pestaña "**Examinar**" (Browse) y busca el archivo **LibreriaVisualFinal.dll** generado en el Paso 5.
5. **Reconstruye la Solución** para asegurar que la DLL esté cargada.

7. IMPLEMENTACIÓN EN LA VENTANA PRINCIPAL

1. Abre el MainWindow.xaml de tu proyecto CRUD.
2. **Declaración del Namespace:** Añade la declaración usando el **Mapeo XMLNS** que definiste (Paso 4):

XML

xmlns:final="http://miscontroles.com/final"

3. Implementación y Conexión:

- Añade el BotonVisual y el VisorProgreso en un contenedor (Grid o StackPanel).
- **Asigna un nombre (x:Name)** al VisorProgreso (ej., x:Name="miVisor").
- En el Code-Behind de MainWindow.xaml.cs, suscribe el evento Click del BotonVisual. Dentro de ese manejador de eventos, **accede al VisorProgreso usando su x:Name** para actualizar sus DPs (miVisor.MensajeAccion = ... y miVisor.ProgresoValor = ...).

ENTREGA

INSTRUCCIONES DETALLADAS DE ENTREGA

La entrega debe ser **una única carpeta comprimida** (formato .zip o .rar) que me permita abrir y ejecutar tu proyecto CRUD (Proyecto 2) inmediatamente y verificar el código fuente de tu librería (Proyecto 1).

Elemento a Entregar	Contenido Requerido	Propósito de la Entrega
Proyecto 1: Carpeta Fuente de la Librería	La carpeta completa del proyecto LibreriaVisualFinal . Esto incluye todos los archivos de código (el .cs, el XAML) y la carpeta /bin/Debug que contiene el archivo LibreriaVisualFinal.dll.	Permite verificar la correcta implementación del UserControl y el CustomControl en su código fuente, y asegura que la DLL esté disponible para el otro proyecto.
Proyecto 2: Carpeta Fuente de la Aplicación CRUD	La carpeta completa de tu proyecto CRUD (UD2). Es fundamental que esta carpeta contenga la referencia funcional a la DLL. Será el proyecto que abriré para ejecutar la aplicación.	Es el proyecto que abriré para ejecutar la aplicación y comprobar visualmente que los componentes funcionan correctamente.
Archivo README (Texto)	Un archivo de texto o Word que contenga: 1. Tu nombre completo. 2. Dos capturas de pantalla de la aplicación CRUD mostrando los componentes funcionando. 3. Justificación de la compatibilidad: El .NET Framework exacto (ej., 4.7.2) usado para crear la librería.	Documentación obligatoria para la calificación.

Importante: Asegúrate de compilar la solución completa antes de comprimir la carpeta final. Si tu proyecto CRUD (Proyecto 2) compila y ejecuta correctamente en tu máquina al abrirlo, significa que todos los archivos necesarios para la DLL están donde deben estar para que yo pueda corregirlo.

CRITERIOS DE EVALUACIÓN

Criterio	¿Qué debes demostrar?	Puntuación
Integración Básica y Framework	Tu proyecto CRUD debe compilar sin errores al añadir la DLL. Debes demostrar que la configuración de .NET Framework 4.x fue exitosa y que la DLL se cargó.	25%
El Visor de Progreso (UserControl)	Has creado el UserControl (VisorProgreso) correctamente con una barra de progreso y una etiqueta, y has expuesto las dos Dependency Properties necesarias.	20%
El Botón Visual (Custom Control)	Has creado el CustomControl (BotonVisual) usando el Generic.xaml y has implementado la lógica para que cambie de color (aleatorio o alternado) al hacer clic.	25%
La Conexión (Coordinación)	Al hacer clic en el botón, el visor debe actualizarse inmediatamente con el mensaje de la hora actual y la barra de progreso debe reflejar un valor aleatorio .	25%
Profesionalismo (Mapeo)	Has utilizado el Mapeo XMLNS (xmlns:final="...") en lugar de la sintaxis larga (clr-namespace).	5%
TOTAL		100%